

Project Contract — Shared Interfaces

Project: Agentic AI for Telemetry Analysis on AROL Capping Machines

Status: PROPOSAL from Person A — to be reviewed and agreed with Person B before coding begins.

This file defines the three things our two halves must agree on so they fit together at integration: the **event-table schema**, the **tool signature convention**, and the **config layout**. Everything here is grounded in the actual telemetry files we were given, not the brief's simplified example. Items marked **OPEN** need a joint decision — don't treat them as settled.

What the real data looks like (context for every decision below)

We inspected the actual telemetry files before writing this. The key facts:

- One file = **one machine** (`MCC777...`), one day, **86,400 rows** (one sample per second), ~57 MB.

- **36 heads** (not 48). Columns per head: `H## Count` , `H## AppTorque` , `H## Status` , plus a single `timestamp` . 109 columns total.
 - **Counters are clean:** they only hold (+0) or increment by exactly **+1**. No wraps, no resets, no decreases anywhere in a full file.
 - **Counters continue across day-files:** a day ends and the next begins at the same count. Files in a pool must be processed **in timestamp order as one continuous stream**.
 - A closure's `AppTorque` and `Status` sit on the **same row** as the +1 increment — no need to look at the previous row.
 - Only **three status codes** appear in the data: `0` , `2` , `65` .
 - `0` → successful closure, torque ≈ 2.0 Nm (a real cap applied).
 - `2` → **No Load**, torque ≈ 0.0 — the head cycled but no cap was present. **Not a failure — the absence of a cap.**
 - `65` → **Bad Closure** (reject), very rare (single-digit occurrences per file).
 - Data is otherwise clean: no missing cells, no duplicate timestamps, essentially regular 1-second spacing.
-

1. Event-table schema

One row = one closure event (one count increment on one head). This is the artifact Person A produces and Person B's agent consumes.

column	type	source	notes
<code>ts</code>	datetime (UTC)	timestamp of the increment row	files processed in order and stitched across days
<code>machine_id</code>	string	parsed from filename (MCC777...)	one machine today, but pools may differ — carry it so multi- machine works later

column	type	source	notes
head_id	string	the head's column prefix	keep the H## form: H01 ... H36
torque	float	H## AppTorque on the increment row	unit: Nm
status	int	H## Status on the increment row	raw code (0 / 2 / 65) – keep the raw number
error_class	string	decoded from status	Closure OK , No Load , Bad Closure
reject_signal	bool	derived from status	true for 65; false for 0 and 2

column	type	source	notes
cap_present	bool	derived from status	true when a cap was actually applied (status 0 or 65); false for No Load (2)

Status decoding (only the codes that actually appear)

status	error_class	reject_signal	cap_present
0	Closure OK	false	true
2	No Load	false	false
65	Bad Closure	true	true

Keep the decoder **extensible** — the brief lists more codes (3, 4/5, 8/9, 16/17, 32/33, 64) that don't appear in our pools but could in a grading dataset. Map any unseen code to `error_class = "Unknown (<code>)"` and flag it, rather than crashing.

OPEN — needs a joint decision (and worth raising with the professor)

The success-rate denominator. A "No Load" (status 2) is not a failed closure — no cap was there to close. If we count No-Load cycles as failures, every success rate is wrong and looks terrible. Proposal: **success rate = successful closures (status 0) ÷ cap-present closures (status 0 or 65)**, excluding No-Load from the denominator. The `cap_present` column exists precisely so this filter is one clean step. **Person B: does your reporting assume a different denominator? Let's lock this together — it drives most of the KPI numbers.**

OPEN — for Person B

Does the agent need any other per-event field to answer its report queries — e.g. a line ID, a shift/batch marker, or a derived "hour of day" for time-of-day analysis? If so, name it now so it's in the schema from the start.

2. Tool signature convention

Every analysis function Person A writes is called by Person B's agent. They all share one shape so the agent can call any of them uniformly.

Input

```
def tool_name(events: pd.DataFrame,
**params) -> dict:
    ...
```

- First argument is always the **event table** (the schema above).
- Everything else is keyword params (filters, window sizes, thresholds).

Output — a consistent envelope

```
{
  "result": { ... the actual answer:
numbers, tables, flagged rows ... },
  "meta": {
    "n": <int>,                # sample
size the result is based on
    "filters_applied": [ ... ],  # e.g.
["cap_present=True", "head_id=H05"]
    "notes": "<optional caveats>" # e.g. "n
low; not significant"
  }
}
```

- **Always a dict**, never a printed string or bare number — this is the rule that lets the agent consume every tool identically.
- **JSON-serializable only**: cast numpy types to plain `float` / `int` before returning. (numpy types

silently break JSON – the agent will choke on them.)

- `meta.n` is mandatory – the confidence/limits section of every report depends on it.

Error / empty handling

When a tool can't produce a result (no matching data, bad params), it **returns** rather than throws:

```
{ "result": None, "meta": {...}, "error":  
  "no cap-present closures for head H12 in  
  window" }
```

OPEN – for Person B

You own the agent that consumes these dicts. **Does this envelope fit your report assembler, or do you want `result / meta` shaped differently?** You can see the consumer side; shape it however makes your dispatch and reporting cleanest – then Person A builds every tool to match.

3. Configuration

All paths, thresholds, and parameters live in one config file – nothing hard-coded (a brief requirement).

Proposed `config.yaml`

```

data:
  pools:
    feb: ["path/to/telemetry_...2026-02-01.csv", "path/to/...2026-02-02.csv"]
    mar: ["path/to/...2026-03-01.csv", "path/to/...2026-03-02.csv"]
    apr: ["path/to/...2026-04-01.csv", "path/to/...2026-04-02.csv"]
    n_heads: null          # null = auto-detect from columns (preferred over hard-coding 36)

analytics:
  torque_expected_min: 1.5    # tune from data
  torque_expected_max: 2.5
  drift_window_seconds: 3600
  idle_window_seconds: 300    # sustained
  No Load = idle
  anomaly_sigma: 3.0

agent:
  # Person B's keys live here

```

- A **pool** is an ordered list of day-files for one machine, processed as one continuous stream.
- **Auto-detect head count** from the columns rather than hard-coding 36 or 48 — the grading dataset may differ.
- Sections keep ownership clean: `data` / `analytics` are Person A's, `agent` is Person B's, one shared file.

OPEN — for Person B

One shared `config.yaml` with sections (proposed), or separate files? Proposal is one file so there's a single source of truth.

Cross-cutting agreements

These aren't "sections" but they're part of the contract:

1. **Pool = ordered, stitched stream.** Files in a pool are processed in timestamp order and treated as one continuous counter history. Closures that straddle a day boundary must still be caught. Both sides assume this.
2. **The placeholder handshake.** Person B builds the agent against a synthetic event table (matching this schema exactly) while Person A builds the real pipeline. Because both code to this schema, swapping the real pipeline in later is painless. **The synthetic generator must emit exactly these columns/types, and use an auto-detected or 36-head layout.**
3. **Schema changes are a two-person decision.** If either side needs to change the event-table shape mid-project, it's agreed and edited **here**, in this file. A silent schema change is the most common thing that breaks a parallel two-person build.

4. Repo layout (proposed):

```
src/ingestion/    # Person A: loader,
validation, closure detection, event
table
src/analytics/    # Person A: stats,
trend, anomaly, correlation
src/agent/        # Person B: agent loop,
dispatch, report assembler
src/interface/    # Person B: CLI,
export, plots
tests/
docs/             # schema.md,
methods.md, this contract.md
config.yaml
```

Sign-off

- ☐ Person A has read and agrees
- ☐ Person B has read and agrees
- ☐ The three **OPEN** items (success-rate denominator, extra event fields, output envelope) are resolved
- ☐ This file is committed to `docs/contract.md`

Once signed off, Milestone M0 is complete and solo building (Steps 4–14) can begin.